

Deep Research Report on Upgrading an NRL Match Prediction Pipeline

Executive synthesis for your constraints

A pipeline that treats bookmaker markets as a strong, auditable prior, then learns leak-safe residual structure from your existing feeds, is the most reliable way to improve accuracy without adding paid data or breaking off-season robustness. Betting odds routinely set a high bar for match forecasting, and several peer-reviewed studies argue for combining markets with statistical models rather than trying to “beat” markets head-on. ¹

A concrete architecture that fits your current stack and data volume is:

- **Tier A: Structural team-strength model (dynamic, hierarchical, small feature set)** that produces baseline scoring intensities (and a full score distribution) with strong early-season behaviour through partial pooling and time evolution. Dynamic team attack/defence models with Poisson scoring are standard in football score modelling, including state-space variants with stochastically evolving strengths. ²
- **Tier B: LightGBM residual model(s)** that predict multiplicative adjustments to Tier A intensities from your engineered covariates (rest, travel proxies from fixtures, Origin flags, ladder deltas, form deltas, etc.), keeping the target distributional and leak-safe. LightGBM is explicitly designed for high-dimensional, sparse feature spaces via techniques like Exclusive Feature Bundling (EFB). ³
- **Tier C: Probabilistic calibration + ensemble/stacking** for match outcomes (and optionally line cover), using out-of-fold predictions only. Post-hoc calibration is often necessary for boosted trees used as probability engines, and well-studied methods (isotonic, Platt/logistic, beta calibration) can materially improve probability quality under proper scoring rules. ⁴

Two research threads strongly support this design choice in your setting:

1. **Markets encode information you cannot infer from your feeds**, and combining odds with historical-score models is an established strategy. Egidi, Pauli, and Torelli propose a Bayesian Poisson score model in which scoring rates combine information from historical results and bookmaker odds, and they explicitly derive “implicit scoring rates” from the three-way odds via the Poisson-difference (Skellam) construct. ⁵
2. **Possessive ball-sports like rugby league can deviate from simple independent Poisson assumptions**, especially when the restart rule and scoring frequency induce dependence. Singh, Phil Scarf ⁶, and Rose Baker ⁷ place rugby league within “possessive ball-sports” and argue that Poisson-type models are natural when scoring events are rare or restarts are contested, while other restart rules can require richer two-team score dependence. ⁸

The remainder of this report maps that architecture onto your specific constraints (Final-row training, Pre-game inference; R+Python+SQLite; no new paid feeds; offseason failure tolerance), compares model candidates, and gives an integration roadmap.

Candidate model families for score and outcome prediction

Independent Poisson as the baseline you already have

Independent Poisson score models (estimating a mean for home and away score separately, then deriving outcome probabilities via simulation) remain a common baseline in sports score forecasting. The Maher-style structure models each team's score as a Poisson variable with intensity driven by team attack and opponent defence plus home advantage (and later extensions). ⁹

Your current design (two LightGBM Poisson regressors; Poisson simulation for outcomes) matches this baseline but with a much richer covariate set and non-linear function class. LightGBM explicitly supports a Poisson objective. ¹⁰

Limitations in your context (NRL points): - Independence between home and away scores is often false in two-team sports (pace, game state, officiating style, fatigue). Classical football work explicitly tests and relaxes independence, at least for specific scorelines. ¹¹

- Poisson variance equals mean; points in higher-scoring codes can show overdispersion and structured tails, which can break both score interval calibration and derived win probabilities. Work on high-scoring Australian Football League ¹² explicitly warns that Poisson/Skellam modelling of scores can be problematic due to overdispersion in high-scoring settings. ¹³

Bivariate Poisson and Dixon-Coles style dependence corrections

Bivariate Poisson via trivariate reduction models home score X and away score Y as $X = X_1 + X_3$, $Y = Y_1 + Y_3$ (independent Poisson components), giving **positive covariance** $\text{Cov}(X, Y) = \lambda_3$. ¹⁴

This is directly implementable on top of your current pipeline because you already produce $\hat{\mu}_h$ and $\hat{\mu}_a$; you can estimate λ_3 globally (or by match-type buckets) and simulate correlated scores cheaply.

Dixon-Coles add two practically relevant ideas: - a dependence correction $\tau_{\lambda, \mu}(x, y)$ that perturbs probabilities for the low-score cells (notably 0-0, 0-1, 1-0, 1-1) while preserving Poisson marginals, capturing the empirically observed misfit of pure independence in those regions. ¹⁵

- exponential down-weighting of old matches via $\phi(t) = \exp(-\xi t)$ so team strength estimates track recent form. ¹⁶

Fit to NRL constraints: - The low-score cell correction matters less for rugby league points than for low-scoring soccer, but the broader point stands: local dependence can matter even if global correlation is modest. ¹⁷

- Time-decay is easy to implement leak-safely (weights depend only on time), but you already carry rolling form and ELO, so decay may be redundant unless it replaces brittle high-dimensional sparsity. ¹⁸

Dynamic hierarchical Bayesian attack/defence models

Dynamic models treat team strengths as latent states that evolve. Koopman and Lit specify bivariate Poisson match outcomes and let attack and defence strengths vary over time with an autoregressive state process; intensities depend on current attack/defence and a home advantage term. ¹⁸

This directly addresses early-season and offseason stability: priors plus evolution control how quickly the model moves away from last season.

Egidi et al. also use a hierarchical Bayesian Poisson score model and build in dynamics across seasons (attack/defence effects evolve), then incorporate bookmaker information through a convex combination mechanism (detailed below). ¹⁹

Dynamic Bayesian models are also proven in Australian high-scoring codes: Manderson, Kevin Murray ²⁰, and Berwin A. Turlach ²¹ fit a dynamic Bayesian forecasting model using the Skellam distribution for Australian Football League ¹² match results in Stan ²², explicitly motivated by the fact that team scoring/defence changes over time. ²³

Fit to NRL constraints: - Your sample size ($\approx 2.6k$ matches) is well within what Stan/PyMC can handle for team-level latent states. ²⁴

- Dynamic partial pooling is the cleanest early-season prior you can get without new data: it formalises “we know last year matters, but not perfectly.” ²⁵

- It also gives you a coherent uncertainty model for offseason (wide priors; slow drift), which your current GBDT-heavy design will not naturally provide. ²⁶

Gradient-boosted trees plus calibration

LightGBM’s GOSS and EFB explicitly target situations like yours: many predictors, many sparse ones, with the need to train efficiently while maintaining accuracy. ³

LightGBM supports a Poisson objective for count-like non-negative targets. ²⁷

However, tree ensembles often need **post-hoc calibration** when you use them to produce probabilities (directly or via downstream simulation). Niculescu-Mizil and Rich Caruana ²⁸ document calibration behaviour across learning algorithms and show that calibration methods like Platt scaling and isotonic regression can materially change probability quality. ²⁹

Kull, Silva Filho, and Peter Flach ³⁰ propose **beta calibration** and show improvements (including for boosted models) on log-loss and Brier score, using internal CV to fit the calibrator without leakage. ³¹

Fit to NRL constraints: - You already have LightGBM and blocked CV. Adding an explicit calibration stage is a high-return, low-risk change because it does not alter your feature computations or leak-safety boundaries. ³²

- Calibration should target the thing you actually consume (win probability, cover probability), not just the Poisson score means. ³³

Hybrid and ensemble designs using odds, ELO, and ML

Two strong research signals support explicit odds inclusion:

- Wunderlich and Memmert present a betting-odds-informed Elo rating model and argue that betting odds contain substantial pre-match information; their Elo-odds variant outperforms classic Elo variants, supporting the idea that markets encode more than results alone. ³⁴

- Egidi et al. incorporate bookmaker information directly into a Bayesian score model by deriving market-implied “scoring rates” and then using convex combinations between historical-based intensities and bookmaker-based parameters. ³⁵

Your own data sources (fixtures/results + team performance + ladder + H2H + line) are enough to build an odds-informed ensemble without new providers, but you must handle bookmaker margin/overround. The R package **implied** documents multiple vig/overround removal methods, including basic normalisation, power methods, and Shin-type procedures. ³⁶

NRL-specific validation that odds are informative exists even outside pre-game modelling: Guan et al. develop in-game win probabilities for the NRL and explicitly use betting odds as a key input to inform win probabilities, indicating that odds supply signal even when richer in-game features exist. ³⁷

Feature engineering upgrades using only your current feeds

Sparse feature handling that stays leak-safe

Your current predictor count (~355) with “highly sparse” features suggests at least one of: high-cardinality categoricals (venues, referees, matchups), indicator explosions (team-by-venue-by-season), or interaction terms that create low-support columns.

Three changes improve stability without adding new data:

1. **Collapse sparse indicators into hierarchical priors rather than raw one-hots** when the category count is large and per-category data is thin. Tree methods can memorise rare indicators, and the risk is amplified when categories are season-specific. LightGBM’s own motivation highlights sparse, one-hot-like spaces as a common target, but the model-expressive power also makes leakage-like memorisation easier if your CV splits have related examples. ³⁸
2. **Use category encodings that are explicitly leakage-aware** (out-of-fold target statistics), or avoid target-based encodings unless you can guarantee they are computed strictly from the training fold history. CatBoost’s core motivation is that naive target statistics and naive boosting can induce a prediction shift via target leakage, and it proposes ordered procedures to address it. ³⁹
3. **Bundle features by team-vs-opponent deltas rather than separate home and away blocks** to reduce redundancy and sparsity. This is not “dimensionality reduction” in the abstract; it makes the model’s job simpler because the same semantic feature (e.g., “recent attacking form gap”) exists regardless of home/away role.

Practical engineering rule for your pipeline: if a feature has <~30 historical non-missing/non-zero observations across 13 seasons, either (a) move it into a pooled latent effect (Bayesian tier), or (b) remove it unless you can justify it as a deterministic rule-based flag (e.g., Origin week) with stable meaning.

Early-season priors and missingness treatment

Early-season instability is a predictable failure mode when you engineer rolling form features: teams have few within-season matches, so rolling means either collapse to small-sample noise or require ad hoc “season-to-date” defaults.

Dynamic hierarchical models solve this cleanly by construction: attack/defence effects evolve from previous-season states with a controlled innovation variance (or AR coefficient). Koopman and Lit explicitly model time variation in attack/defence strengths with an autoregressive evolution. ¹⁸

Egidi et al. also specify season-to-season evolution terms for team effects and explicitly treat the first season differently, reflecting the need for initial priors. ⁴⁰

In your current GBDT-heavy system (without a latent state tier), you can still get most of the benefit by forcing **shrinkage-to-prior** on rolling features:

- For each team metric m , create a prior mean m_0 from last season (or last N seasons weighted), and a “sample size” n .
- Define a shrunk estimate $\tilde{m} = w \cdot m_{\text{observed}} + (1 - w) \cdot m_0$ with $w = \frac{n}{n+k}$, where k is a tunable pseudo-count controlling how many games you need before you trust current-season form.
- Store both $\tilde{m}$ and w (or n) as features; the model can learn that early-season values are less reliable.

Missingness should be explicit, not silently imputed:

- Keep LightGBM’s native missing handling.
- Add “is_missing” flags for market features (odds absent), performance feed outages, and rare-team stats. This supports your “missing dependency” requirement because the model can learn a distinct regime for absent markets rather than alternative values implicitly meaning “market missing.” ⁴¹

Market-derived features from head-to-head and line

The first job is **vig removal**. Basic normalisation (divide inverse odds by their sum) is the simplest and most common, but it bakes in the assumption that margin is distributed proportionally, which is often false under favourite-longshot bias. ³⁶

You can compute multiple “fair probability” estimates and let the model select:

- **basic normalisation** (multiplicative) ⁴²
- **power method** ³⁶
- **Shin-style** estimates (in practice, you can use the implementation in implied; the underlying insider-trader model traces back to H. S. Shin ⁴³’s state-contingent claim pricing work). ⁴⁴

From your available market feed types:

- **H2H odds**: produce
- fair home win probability p_{mkt}
- market log-odds $\log \frac{p}{1-p}$
- implied information proxy: entropy $-\left[p \log p + (1 - p) \log(1 - p)\right]$ (high entropy = tight game).
- **Line odds** (spread and prices): produce
- market-implied probability of covering (after vig removal)
- implied spread (points) as a direct feature
- “line disagreement” features: compare your current model’s predicted margin distribution to the market line (e.g., z-score of line under your predicted margin variance).

A research-backed route to use markets *as priors*, not merely features, comes from Egidi et al.:

1. Derive market-implied scoring intensities by solving for $(\hat{\theta}_1, \hat{\theta}_2)$ that reproduce the market three-way probabilities under a Poisson-difference (Skellam) model. ⁴⁵
2. Combine historical-based intensities and market-based parameters by convex combination, estimated within a Bayesian model. Their Equation (3.3) is exactly that: Poisson rates become mixtures weighted by parameters p_m , which they assign Beta priors to. ⁴⁶

NRL does not have a large draw rate, so the three-way mapping is less central than in soccer. You can still apply the same idea in a two-way setting: treat the market-implied win probability (or cover probability) as a strong prior for the match outcome and allow your model to move it only where your features have consistent historical justification.

Home/away interaction and delta features

NRL modelling benefits from “relative strength” features because they are stable across season changes and reduce feature explosion.

Concrete deltas you can compute using your current feeds:

- **attack_delta**: (home rolling points-for) – (away rolling points-against)
- **defence_delta**: (home rolling points-against) – (away rolling points-for)
- **ladder_delta**: (home ladder points, points differential, rank) – (away equivalents)
- **rest_delta**: home days since last match – away days since last match
- **travel_proxy_delta** (if venue is in fixtures): home state vs away state flags are crude but can still capture systematic patterns; if you avoid new data, keep this minimal and defensible.

These are not “nice to have.” They reduce the model’s need to learn home/away symmetry twice (once for home score and once for away score) and lower the risk that sparse interactions dominate.

Evaluation and backtesting that isolates true gains

Proper scoring rules for probabilities and scores

Proper scoring rules incentivise honest probabilistic forecasts and provide a rigorous basis for model comparison. ³³

For your outputs, evaluate at three levels:

Match outcome probabilities - Log loss (logarithmic score) for home win probability. Log score is strictly proper and punishes overconfident wrong calls. ³³

- **Brier score** for probability accuracy; consider decomposition (reliability vs resolution) when diagnosing changes. ⁴⁷

Score distribution quality - Log score over the joint score distribution (or over the implied margin distribution) when you can compute probabilities for each scoreline or margin bin. This directly measures whether your simulated distribution matches realised outcomes. ³³

- **CRPS** for points and margin distributions. `scoringRules` provides CRPS (and log score) implementations for Poisson and negative binomial families, and supports both parametric distributions and simulation-draw evaluation. ⁴⁸

Decision-market alignment diagnostics - Line cover probability calibration if you use line markets operationally.

- **Market-relative log loss**: compare your model's log loss to the vig-removed market probability baseline.

Calibration diagnostics that matter operationally

Calibration is not optional if you act on probabilities. Gneiting and Adrian Raftery ⁴⁹ define calibration and sharpness and give diagnostic tools like probability integral transform histograms and calibration plots. ⁵⁰

For binary win probabilities, use:

- Reliability curves (bin predicted probabilities; plot observed win rate).
- ECE/MCE as a compact summary, but treat them as diagnostics, not objective functions. ⁵¹
- For score distributions: PIT-like diagnostics on discrete distributions (either randomised PIT or distributional checks).

Time-aware validation protocol

Your in-season round-blocked CV is directionally correct. Two requirements tighten it:

1. **Split by season and round with expanding windows**, not random folds.
2. **All feature engineering must run "as-of" the prediction date** inside each fold (no global precomputation that peeks).

Bergmeir, Rob J Hyndman ⁵², and Koo discuss cross-validation in time series settings and the pitfalls of naive splitting under dependence and non-stationarity; at minimum, your protocol must preserve temporal order. ⁵³

A robust backtest design for your dataset size:

- Outer loop: **leave-one-season-out** (13 folds).
- Inner loop: within each training set, tune hyperparameters using rolling-origin blocks by round (your existing design), but only after confirming that fold-level feature computation is leak-safe.

This protocol also naturally supports offseason robustness tests: treat "season start" as a separate regime and score it separately.

Ablation plan that attributes lift

Ablation must reflect your actual decision outputs, not just point MAE.

Minimal ablation ladder:

1. Market-only baseline (vig-removed H2H and line cover probabilities). ³⁶
2. Elo-only baseline (your current Elo).
3. Your current LightGBM Poisson (as implemented).
4.
 - calibration only (no other changes). ³²
5.
 - market features in the model (still calibration).
6.
 - structural Bayesian team-strength tier feeding LightGBM residuals. ⁵⁴
7.
 - bivariate dependence (bivariate Poisson or Dixon–Coles style correction). ⁵⁵

Store per-ablation out-of-sample predictions in SQLite so you can compute deltas on the same match set, avoiding evaluation drift.

Recommended model architecture for your stack

Core design

The proposed system produces **a calibrated win probability and an explicit score distribution** while remaining robust to missing odds feeds.

Component 1: Dynamic hierarchical team-strength model (baseline rates)

- Output: baseline μ_h , μ_a , and optionally baseline distribution parameters for overdispersion.
- Inputs: only identity-level information always present (home team, away team, season, round, possibly venue/home indicator).
- Mechanism: attack/defence latent variables with partial pooling and time evolution (AR or random walk). Koopman & Lit show a clear template: bivariate Poisson match outcomes; intensities depend on attack/defence and a constant home advantage; attack/defence evolve autoregressively. ¹⁸
- Optional: incorporate market priors when available using convex combinations inspired by Egidi et al. (Equation 3.3) so the market shifts the baseline rates but does not overwrite learned structure. ³⁵

Component 2: LightGBM residual model (covariate effects without destroying identifiability)

- Target: final home points and away points, but model it as an adjustment to the baseline: $-\log(\lambda_h) = \log(\mu_h) + f_h(X)$
- $-\log(\lambda_a) = \log(\mu_a) + f_a(X)$ - You keep LightGBM's Poisson objective. ⁴¹
- Practical constraint: regularise aggressively and cap depth because your row count is small relative to feature count.

Component 3: Joint score dependence layer

Two options, both compatible with your Poisson simulation pathway:

- **Bivariate Poisson shared component:** estimate λ_3 (global or bucketed) so $X = X_1 + X_3, Y = Y_1 + Y_3$, giving $\text{Cov}(X, Y) = \lambda_3$. ¹⁴
- **Dixon–Coles low-score perturbation:** apply $\tau_{\lambda, \mu}(x, y)$ correction for low-score cells if the data show systematic misfit; even if NRL scores are higher on average, the principle is to correct structured dependence without rewriting the entire model. ¹⁵

Given rugby league scoring distributions and the possibility of negative dependence in certain match states, start with the shared-component bivariate Poisson (simple, stable), then only escalate if diagnostics show dependence structure not addressed. ⁵⁶

Component 4: Calibration and stacking

- Build win probability from the joint score distribution (simulation or exact).
- Calibrate the resulting win probability using out-of-fold predictions. Beta calibration is a strong parametric choice and has empirical support across models and datasets under both log loss and Brier. ³¹
- If you run multiple forecasters (market, Bayesian tier, LightGBM tier), combine them with a stacking model trained on cross-validated predictions only. The Super Learner literature formalises weight selection by cross-validation and provides oracle-type guarantees under conditions. ⁵⁷

Offseason and missing dependency robustness

Hard requirements you can enforce mechanically:

- If odds are missing:
 - Tier A and Tier B still produce rates and win probabilities.
 - Tier C stacking re-normalises weights over available forecasters (or falls back to a default weight schedule).
- If performance feed is missing:
 - Tier B drops to a reduced feature set (rest, ladder, Elo, season/round).
 - The model still runs, just with less lift.

This is easier with the tiered design because Tier A uses only fixtures/results identities, which you already have.

Where this differs from your current system

Your current model treats all engineered signals as peers. The recommended upgrade explicitly separates **stable team strength** (latent, dynamic, pooled) from **situational modifiers** (rest, Origin, short-term form) and from **market priors** (odds). That separation lowers variance, improves early-season behaviour, and makes outages survivable. ⁵⁸

Implementation roadmap and concrete upgrade plan

Quick wins

1. **Add a formal calibration stage for win probabilities**
2. Use out-of-fold win probabilities from your current Poisson simulation and fit beta calibration (or isotonic if you prefer non-parametric but data-hungry). ³²
3. Store calibrator parameters per season fold and apply only after training.
4. **Add vig-removed market features (multiple methods)**

5. Compute fair probabilities from H2H odds using at least: basic normalisation and power method; optionally Shin. ⁵⁹

6. Treat these as baseline predictors and as features.

7. Enforce “as-of” feature computation at fold time

8. Persist fold definitions and run feature computation per fold.

9. This removes the most common silent leakage in sports pipelines: aggregate stats computed using full-season context but used to train earlier rounds. ⁶⁰

10. Replace raw sparse indicators with pooled effects or drop them

11. Start by logging per-feature support counts and pruning the lowest-support tail.

Medium-term upgrades

1. Bivariate Poisson dependence layer on top of your two Poisson means

2. Fit a global λ_3 (or a small set by discrete regimes like “close market” vs “blowout market”) by maximising bivariate Poisson likelihood given your predicted marginals. ¹⁴

3. Recompute win/line probabilities with correlated simulations.

4. Dynamic hierarchical baseline model (Stan) feeding LightGBM residuals

5. Implement a team-level attack/defence model with season-to-season evolution (random walk or AR(1)). Koopman & Lit give explicit intensity specifications and latent dynamics. ¹⁸

6. Use Tier A log-rate as an offset in Tier B LightGBM.

7. Stacking ensemble across market, Tier A, Tier B

8. Fit a simple constrained logistic meta-model on out-of-fold predictions.

9. Treat Super Learner as the conceptual justification for cross-validated weight selection. ⁵⁷

Advanced (when fundamentals are stable)

1. Market-as-prior on scoring rates (Egidi-style)

2. Derive market-implied rates and combine with historical-based rates by convex combination, estimated within the Bayesian tier. ³⁵

3. This replaces brittle “odds as just another feature” with an interpretable prior mechanism.

4. Possessive-sport dependence models if diagnostics demand it

5. If joint-score dependence in NRL is not well captured by bivariate Poisson’s positive-covariance structure, consider alternatives motivated by possessive-ball-sport restart rules. Singh et al. explicitly place rugby league within this class and discuss when Poisson-match approximations hold. ⁸

6. This is only worth doing after you have calibration and backtesting locked down.

7. Automated model governance and regression tests

8. Store model artefacts, feature schema version, and fold definitions; run leak tests before every retrain.

Expected impact vs complexity

The “lift” values below are deliberately expressed as expected **relative** improvements in proper scoring rules, not accuracy, because proper scoring rules are the right objective for probability forecasts. ³³

Upgrade	Primary metric target	Expected lift (directional)	Complexity	Why it should work
Probability calibration (beta / isotonic)	log loss, Brier	Medium	Low	Beta calibration improves log loss and Brier across settings, especially where raw scores are overconfident. ³²
Vig removal + market features	log loss vs market	Medium	Low	Overround removal is required to interpret odds as probabilities; multiple methods handle favourite–longshot bias differently. ³⁶
Fold-level as-of feature generation	stability across seasons	Medium	Low–Medium	Eliminates silent temporal leakage and gives honest backtests. ⁶⁰
Bivariate Poisson dependence layer	log score on scorelines, margin calibration	Small–Medium	Medium	Captures positive score correlation without changing marginal means. ¹⁴
Dynamic hierarchical baseline (team strengths)	early-season log loss and CRPS	Medium–Large	Medium–High	Latent strength evolution addresses early-season shrinkage and offseason drift. ⁶¹
Stacking market + Bayesian + ML	log loss, Brier	Medium	Medium	Cross-validated weighting improves risk compared with selecting a single learner. ⁵⁷
Market-as-prior on rates (Egidi-style convex mix)	robustness + interpretability	Medium	High	Uses markets as information source in the generative score model itself. ³⁵

Upgrade	Primary metric target	Expected lift (directional)	Complexity	Why it should work
Possessive-sport Markov-type joint model	joint score fit	Uncertain	Very High	More faithful dependence structure but needs careful assumptions without possession data. ⁸

Ranked top-10 action plan with expected lift and effort

1. Add post-hoc calibration on win probabilities (beta calibration first)

Effort: 1–3 days. Lift: Medium. Risk: Low. ³¹

2. Compute vig-removed H2H probabilities using at least two methods (basic + power) and store them

Effort: 1–2 days. Lift: Medium. Risk: Low. ³⁶

3. Make “as-of” feature computation a first-class artefact (per fold, per match date)

Effort: 3–5 days. Lift: Medium (often larger than expected because it removes optimism). Risk: Medium (engineering). ⁶⁰

4. Create a market-only baseline and score it alongside your model every week

Effort: 1 day. Lift: Indirect (measurement). Risk: Low. ⁶²

5. Prune or pool extremely sparse features based on support counts and season stability

Effort: 2–4 days. Lift: Small–Medium. Risk: Medium (can remove helpful niche signals). ³

6. Add a dependence layer (bivariate Poisson shared component) to your score simulation

Effort: 1–2 weeks. Lift: Small–Medium. Risk: Medium (mis-specified correlation sign). ¹⁴

7. Implement a dynamic hierarchical team-strength baseline (Stan) and use it as a log-offset in LightGBM

Effort: 3–6 weeks. Lift: Medium–Large, especially early season. Risk: Medium–High (MCMC operationalisation). ⁶³

8. Stack market, Bayesian baseline, and LightGBM residuals using out-of-fold predictions

Effort: 2–4 weeks. Lift: Medium. Risk: Medium (leakage if stacking uses in-fold preds). ⁶⁴

9. Make missingness regimes explicit (odds missing, performance feed missing) and test them

Effort: 1–2 weeks. Lift: Small (average), Large (robustness). Risk: Low. ⁴¹

10. Adopt a proper-scoring evaluation suite in SQLite (log loss, Brier, CRPS, log score) with per-season reporting

Effort: 1–2 weeks. Lift: Indirect (prevents self-deception, accelerates iteration). Risk: Low. ³³

Leakage checks and failure modes to actively defend against

Leakage is usually not “future final score in a feature.” In sports pipelines it creeps in through aggregation boundaries.

High-risk leakage points in your design:

- **Rolling features computed with global group_by then joined back:** if the computation window inadvertently includes the current match or later matches, your Final rows leak into training. The fix is fold-level “as-of” computation. ⁵³
- **Target-derived encodings** for sparse categoricals: they can leak unless computed on training folds only. CatBoost’s ordered boosting discussion is effectively a warning about this class of leakage/prediction shift. ³⁹
- **Market odds timestamp mismatch:** if your odds feed sometimes includes late moves (near kickoff) and your intended inference snapshot is earlier, you can unknowingly train on information unavailable at prediction time. Guan et al. treat kickoff win probability from betting odds as a conditioning input, which highlights the importance of precise timing. ³⁷

Robustness failure modes:

- **Odds unavailable:** pipeline must still run and default to non-market tiers.
- **Team performance feed delays:** keep a reduced feature set pathway and log which regime fired.
- **Offseason re-start:** your dynamic baseline tier should reset uncertainty appropriately rather than treating Round 1 as “Round 27 continuation.” ²⁵

Integration notes and pseudocode for an R + Python + SQLite pipeline

SQLite schema additions

Keep your existing leak-safe separation (Final rows for training, Pre rows for inference). Add explicit tables for folds, predictions, and model artefacts.

```
-- core feature store
create table if not exists match_features (
  match_id text not null,
  snapshot_type text not null,          -- 'final' | 'pre'
  as_of_utc text not null,              -- feature timestamp
  season integer not null,
  round integer not null,
  home_team text not null,
  away_team text not null,
  y_home integer,                       -- only for snapshot_type='final'
  y_away integer,                       -- only for snapshot_type='final'
  -- ... wide feature columns ...
  primary key (match_id, snapshot_type)
```

```

);

-- explicit time-aware folds (outer and inner)
create table if not exists cv_folds (
  fold_id text not null,          -- e.g., 'loo_season_2018'
  match_id text not null,
  split text not null,           -- 'train' | 'valid' | 'test'
  primary key (fold_id, match_id)
);

-- store out-of-fold predictions by component
create table if not exists oof_predictions (
  fold_id text not null,
  match_id text not null,
  model_key text not null,       -- 'lgb_poisson', 'bayes_tier',
  'market', 'stacked'
  p_home_win real not null,
  mu_home real,                 -- optional
  mu_away real,                 -- optional
  created_utc text not null,
  primary key (fold_id, match_id, model_key)
);

-- model registry / artefacts metadata
create table if not exists model_registry (
  model_key text not null,
  version text not null,
  trained_utc text not null,
  feature_schema_hash text not null,
  training_fold_id text not null,
  notes text,
  primary key (model_key, version)
);

```

R-side data preparation

Key rules:

- Build all “history” features with an explicit `as_of_time` for each match.
- Compute market features with vig removal and store multiple variants.

Pseudocode outline:

```

build_features_asof <- function(fixture_df,
                                results_df,
                                ladder_df,

```

```

        team_perf_df,
        odds_df,
        as_of_time) {
# WHY: centralises "no future data" rule so every feature honours it.
# Filter all inputs to <= as_of_time
results_hist <- results_df[results_df$kickoff_time < as_of_time, ]
ladder_hist  <- ladder_df[ladder_df$as_of_time <= as_of_time, ]
perf_hist    <- team_perf_df[team_perf_df$match_time < as_of_time, ]
odds_hist    <- odds_df[odds_df$as_of_time <= as_of_time, ]

# Rolling team form using results_hist only
team_form <- compute_rolling_form(results_hist)

# Elo updated using results_hist only
elo_tbl <- compute_elo(results_hist)

# Market features: vig removal; store multiple methods
market <- compute_market_features(odds_hist) # basic, power, shin, etc.

# Assemble row-level features for matches whose as_of_time matches this
snapshot
x <- join_all(
  fixtures_df, ladder_hist, team_form, elo_tbl, market,
  key = c("match_id", "home_team", "away_team")
)

# Missingness flags for robustness
x$odds_missing <- is.na(x$p_home_win_basic)
x$perf_missing <- is.na(x$some_perf_feature)

x
}

```

Implementation note: `compute_market_features()` should use the implied package's `implied_probabilities()` with `method = c("basic","power",...)` and store each output in named columns. ⁴²

Python-side modelling

A minimal version of the tiered model that stays close to your current system:

```

def train_lightgbm_with_offset(train_df, feature_cols, y_col, log_offset_col):
    """
    WHY: modelling residual structure on top of a stable baseline reduces
    variance,
    improves early season behaviour, and keeps predictions positive.

```

```

"""
import lightgbm as lgb
X = train_df[feature_cols]
y = train_df[y_col]
offset = train_df[log_offset_col] # log(mu_baseline)

# LightGBM poisson objective exists and is appropriate for non-negative
integer targets.
# (Treat offset by shifting the prediction target in log space.)
y_tilde = y # keep original target; implement offset via init_score or
custom objective

dtrain = lgb.Dataset(X, label=y_tilde, init_score=offset)

params = {
    "objective": "poisson",
    "metric": "poisson",
    "learning_rate": 0.03,
    "num_leaves": 31,
    "min_data_in_leaf": 40,
    "lambda_l1": 1.0,
    "lambda_l2": 5.0,
    "feature_fraction": 0.7,
    "bagging_fraction": 0.7,
    "bagging_freq": 1
}

model = lgb.train(
    params=params,
    train_set=dtrain,
    num_boost_round=500,
    valid_sets=[dtrain],
    valid_names=["train"]
)

return model

def calibrate_win_probabilities(oof_p_home_win, y_home_win):
    """
    WHY: calibrated probabilities improve decision quality and proper scoring
    rules.
    Fit calibration only on out-of-fold predictions to avoid leakage.
    """
    # Option A: beta calibration implementation
    # Option B: isotonic regression (needs more data)
    pass

```


LightGBM supports the Poisson objective explicitly. ⁴¹

If you adopt a more categorical-heavy encoding strategy, consider CatBoost-style ordered procedures to prevent target leakage in encodings. ³⁹

Joint score simulation upgrade (bivariate Poisson shared component)

This keeps your current “Poisson simulation → outcome probabilities” pattern but adds dependence:

```
def simulate_bivariate_poisson(mu_home, mu_away, lambda_3, n_sims=20000,
    rng=None):
    """
    WHY: shared component lambda_3 adds positive covariance between team scores.
    X = X1 + X3, Y = Y1 + Y3, Cov(X,Y)=lambda_3.
    """
    import numpy as np
    rng = np.random.default_rng(rng)

    lam1 = np.clip(mu_home - lambda_3, 1e-6, None)
    lam2 = np.clip(mu_away - lambda_3, 1e-6, None)

    x3 = rng.poisson(lam=lambda_3, size=n_sims)
    x = rng.poisson(lam=lam1, size=n_sims) + x3
    y = rng.poisson(lam=lam2, size=n_sims) + x3

    return x, y
```

The bivariate Poisson construction and its covariance properties are standard. ¹⁴

Market-as-prior variant (advanced)

Egidi et al. derive implicit scoring rates from market three-way probabilities using the Skellam relationship and then mix historical intensities with bookmaker parameters via convex combination. ⁶⁵

You can implement the same idea in a simpler two-outcome NRL setting:

- Convert H2H to vig-removed p_{mkt} . ⁴²
- Convert your model's pre-market predicted win prob p_{mdl} to logit space.
- Fit $\text{logit}(p^*) = w \cdot \text{logit}(p_{\text{mkt}}) + (1 - w) \cdot \text{logit}(p_{\text{mdl}})$ with w learned by season-aware CV.
- Calibrate p^* post-hoc.

This is the “markets as prior” idea, expressed in a way that matches your current outputs and avoids needing three-way draw modelling.

¹ ³⁴ ⁶² The Betting Odds Rating System: Using soccer forecasts to forecast soccer - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC5988281/>

2 18 25 30 54 58 61 63 <https://papers.tinbergen.nl/12099.pdf>
<https://papers.tinbergen.nl/12099.pdf>

3 38 <https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf>
<https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf>

4 29 <https://www.cs.cornell.edu/~alexn/papers/calibration.icml05.crc.rev3.pdf>
<https://www.cs.cornell.edu/~alexn/papers/calibration.icml05.crc.rev3.pdf>

5 19 26 35 40 45 46 56 65 <https://arxiv.org/pdf/1802.08848>
<https://arxiv.org/pdf/1802.08848>

6 14 28 55 <https://pdfs.semanticscholar.org/960b/cb6a5cf11a19baa3db8d341534c4a6154bd2.pdf>
<https://pdfs.semanticscholar.org/960b/cb6a5cf11a19baa3db8d341534c4a6154bd2.pdf>

7 33 50 <https://sites.stat.washington.edu/raftery/Research/PDF/Gneiting2007jasa.pdf>
<https://sites.stat.washington.edu/raftery/Research/PDF/Gneiting2007jasa.pdf>

8 21 <https://orca.cardiff.ac.uk/id/eprint/149591/1/1-s2.0-S037722172200368X-main.pdf>
<https://orca.cardiff.ac.uk/id/eprint/149591/1/1-s2.0-S037722172200368X-main.pdf>

9 22 <https://ideas.repec.org/a/bla/stanee/v36y1982i3p109-118.html>
<https://ideas.repec.org/a/bla/stanee/v36y1982i3p109-118.html>

10 27 41 <https://lightgbm.readthedocs.io/en/latest/Parameters.html>
<https://lightgbm.readthedocs.io/en/latest/Parameters.html>

11 15 16 17 49 https://www.ajbuckeconbikesail.net/wkpapers/Airports/MVPoisson/soccer_betting.pdf
https://www.ajbuckeconbikesail.net/wkpapers/Airports/MVPoisson/soccer_betting.pdf

12 13 23 24 [repository.cam.ac.uk](https://www.repository.cam.ac.uk/bitstreams/eaf99ede-8373-4b25-b823-52e7d5cd6b0f/download)
<https://www.repository.cam.ac.uk/bitstreams/eaf99ede-8373-4b25-b823-52e7d5cd6b0f/download>

20 36 42 44 59 [Introduction to the IMPLIED package](https://cran.r-project.org/web/packages/IMPLIED/vignettes/introduction.html)
<https://cran.r-project.org/web/packages/IMPLIED/vignettes/introduction.html>

31 32 43 52 <https://proceedings.mlr.press/v54/kull17a/kull17a.pdf>
<https://proceedings.mlr.press/v54/kull17a/kull17a.pdf>

37 [sfu.ca](https://www.sfu.ca/~tswartz/papers/AOAS1514.pdf)
<https://www.sfu.ca/~tswartz/papers/AOAS1514.pdf>

39 <https://papers.neurips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.pdf>
<https://papers.neurips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.pdf>

47 https://ore.exeter.ac.uk/articles/journal_contribution/Simplifying_and_generalising_Murphy_s_Brier_score_decomposition/29748851/1/files/56771708.pdf
https://ore.exeter.ac.uk/articles/journal_contribution/Simplifying_and_generalising_Murphy_s_Brier_score_decomposition/29748851/1/files/56771708.pdf

48 [Evaluating Probabilistic Forecasts with scoringRules](https://cran.r-project.org/package=3DscoringRules/vignettes/article.pdf)
<https://cran.r-project.org/package=3DscoringRules/vignettes/article.pdf>

51 <https://proceedings.mlr.press/v70/guo17a/guo17a.pdf>
<https://proceedings.mlr.press/v70/guo17a/guo17a.pdf>

53 60 <https://robjhyndman.com/papers/cv-wp.pdf>
<https://robjhyndman.com/papers/cv-wp.pdf>

57 64 <https://pubmed.ncbi.nlm.nih.gov/17910531/>
<https://pubmed.ncbi.nlm.nih.gov/17910531/>